FIELD NOTES / INFRASTRUCTURE

KUBERNETES SCHEMATIC.

A blueprint-style reference for the objects, control loops, and wiring that make up a Kubernetes cluster — from control plane to pod, drawn out.

SHEETS **01–13** SCOPE **CORE API + NETWORKING + STORAGE** REV **2026.1**

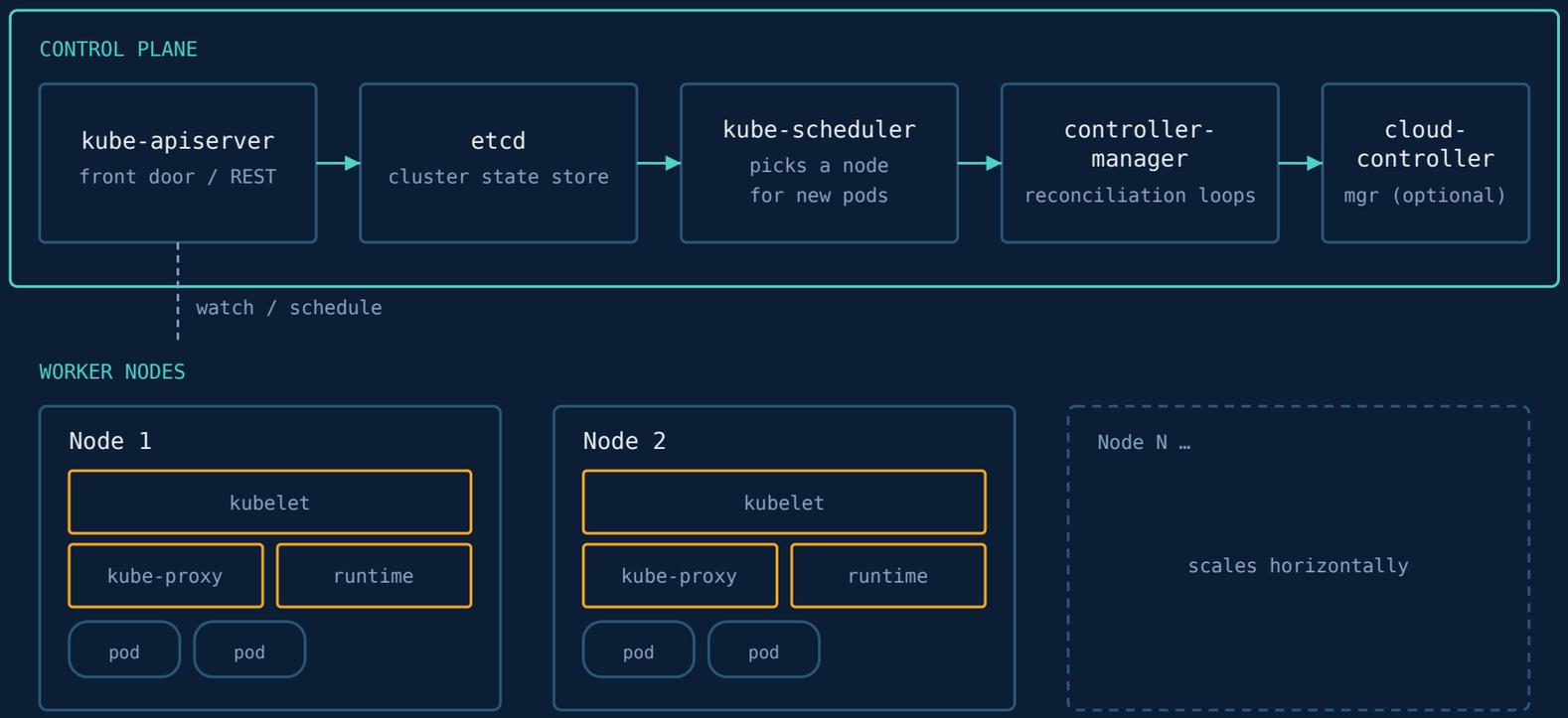
K8S–01 What Kubernetes Is

Kubernetes is a **container orchestrator**: you describe a *desired state* — "run 3 copies of this image, keep this service reachable at this address" — and a set of control loops continuously works to make the *actual state* match it. You don't script deployments step-by-step; you declare an outcome and Kubernetes reconciles toward it, forever, on every node in the cluster.

- **Declarative, not imperative.** You write YAML describing desired state; controllers do the work of getting there and staying there.
- **Self-healing.** If a pod dies, a node fails, or a container crashes, controllers notice the drift and recreate what's missing.
- **Portable.** The same manifests run on a laptop (minikube/kind), a bare-metal cluster, or any major cloud's managed offering (EKS, GKE, AKS).
- **Everything is an object.** Pods, Services, Deployments, ConfigMaps — all are resources stored in one datastore (etcd) and manipulated through one API.

K8S–02 Cluster Architecture

Every cluster splits into two planes: the **control plane** (the brain — decides what should run where) and **worker nodes** (the muscle — actually run your containers).

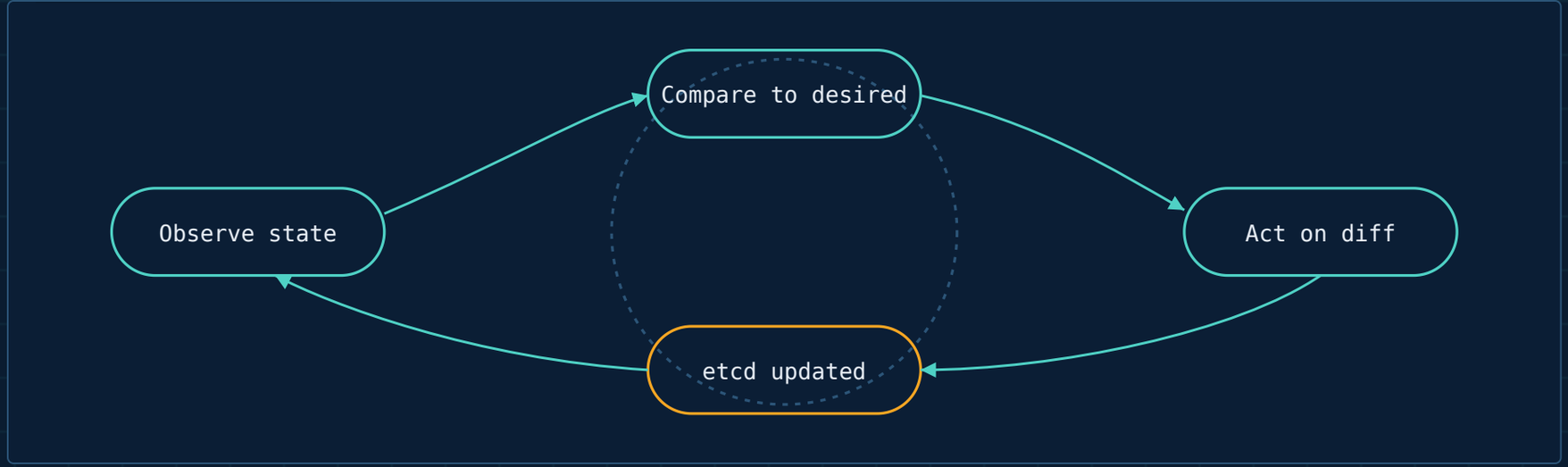


COMPONENT	PLANE	JOB
kube-apiserver	Control	Validates & serves the REST API; every other component talks through it, never directly to etcd.
etcd	Control	Consistent, distributed key-value store holding all cluster state. The single source of truth.
kube-scheduler	Control	Watches for unscheduled pods and binds each to a suitable node based on resources, affinity, taints.
kube-controller-manager	Control	Runs the built-in reconciliation loops (Node, ReplicaSet, Endpoints, Job, etc.).
kubelet	Node	Agent on every node; makes sure the containers described in assigned PodSpecs are running and healthy.
kube-proxy	Node	Maintains network rules on the node so Services resolve and load-balance to the right pods.
Container runtime	Node	containerd or CRI-O — actually pulls images and runs containers via the CRI.

K8S–03

The Reconciliation Loop

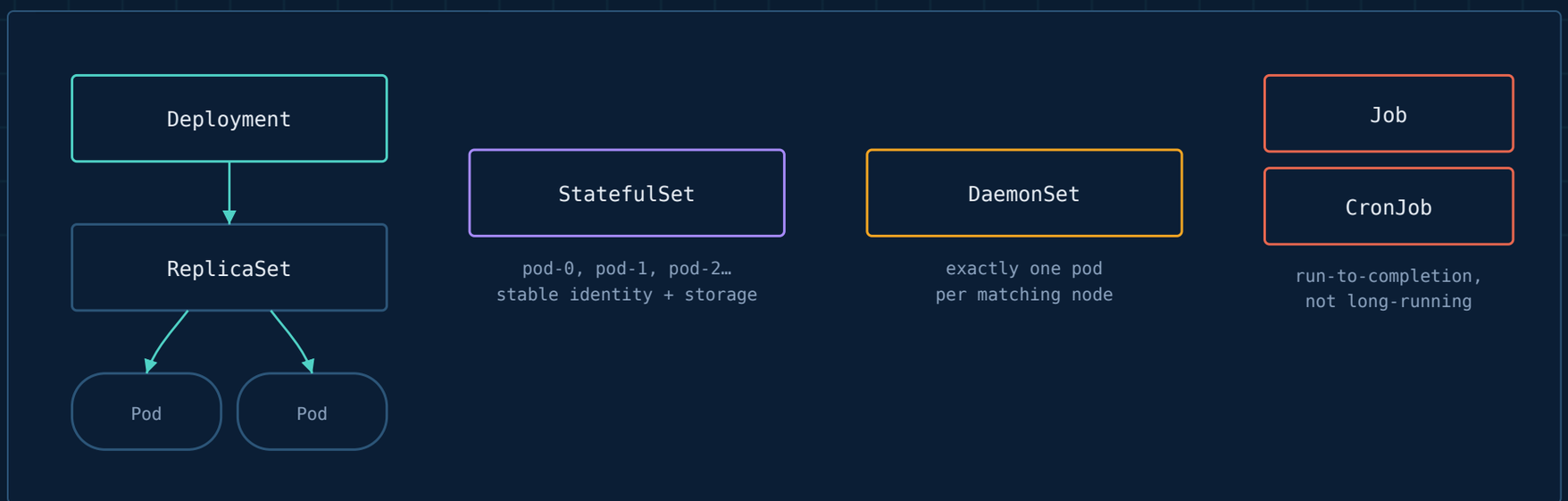
Almost everything in Kubernetes is one instance of the same pattern, running continuously and independently for every kind of object.



A **controller** watches the API server, notices actual state ≠ desired state (e.g. 2 pods running, 3 desired), and issues the calls to close the gap. This same shape drives Deployments, Nodes, Jobs, Services, and every custom controller/operator you'll ever write.

K8S–04 Core Workload Objects

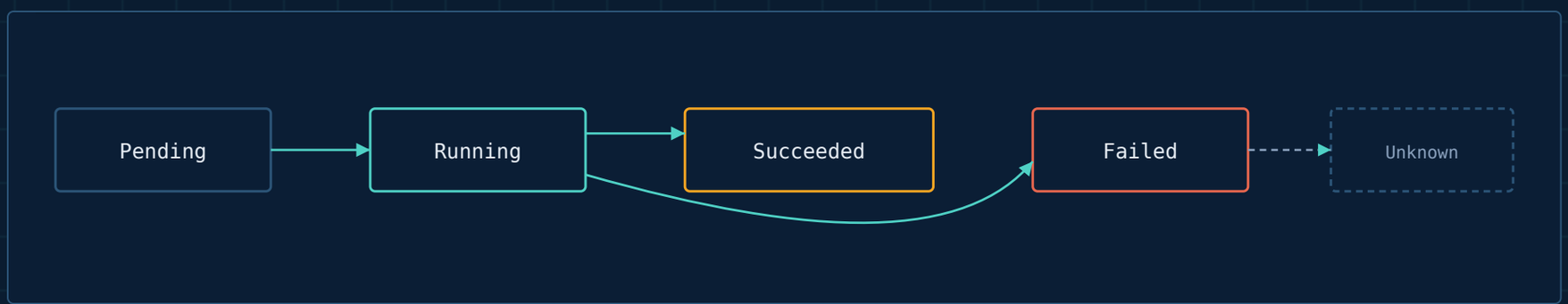
You rarely create bare Pods. Higher-level objects wrap them and add behavior — restart policy, scaling, rollout strategy, or "one per node."



- **Pod** — the smallest deployable unit; one or more containers that share network/storage namespace. Pods are disposable — never edited in place, always replaced.
- **ReplicaSet** — keeps N identical pod replicas running. You almost never write these directly; a Deployment manages them for you.
- **Deployment** — manages ReplicaSets to give you declarative rolling updates, rollback, and scaling. The default choice for stateless apps.

- **StatefulSet** — like a Deployment but for workloads that need stable network identity and stable storage per replica (databases, queues).
- **DaemonSet** — ensures a copy of a pod runs on every (or a filtered set of) node — log collectors, node monitoring agents, CNI plugins.
- **Job / CronJob** — run a pod to completion once, or on a schedule (batch processing, backups, migrations).

K8S–05 Pod Lifecycle & Probes



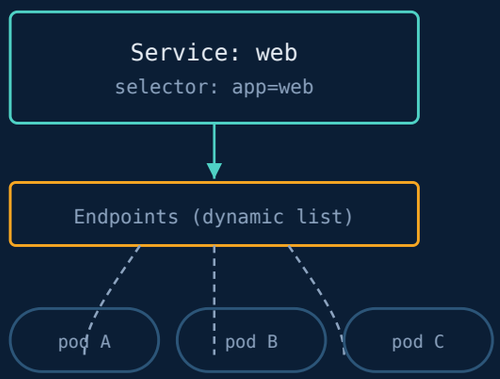
PROBE	QUESTION IT ANSWERS
`livenessProbe`	Is the container still working? Fails → kubelet restarts it.
`readinessProbe`	Can it take traffic right now? Fails → pod pulled from Service endpoints, not restarted.
`startupProbe`	Has slow-starting app finished booting? Blocks the other two probes until it passes.

COMMON TRAP

A missing readinessProbe means Kubernetes sends traffic to a pod the instant its container starts — even if the app inside is still initializing. Always set one for anything user-facing.

K8S–06 Services & Networking

Pods are mortal — IPs change every time one is replaced. A **Service** is a stable virtual address in front of a changing set of pods, matched by label selector.



SERVICE TYPES

ClusterIP
internal only (default)

NodePort
opens a port on every node

LoadBalancer
provisions cloud LB

Ingress
HTTP(S) routing layer
host/path → Service
one entrypoint,
many Services behind it
needs an Ingress
controller installed

TYPE	REACHABLE FROM	TYPICAL USE
ClusterIP	Inside the cluster only	Service-to-service traffic (default type)
NodePort	<NodeIP>:<30000-32767>	Quick external access / dev clusters
LoadBalancer	Public internet, via cloud LB	Production external entrypoints on managed clusters
Ingress	Public internet, via Ingress controller	Host/path-based HTTP routing to many Services with one IP + TLS termination

K8S–07

Configuration: ConfigMaps & Secrets

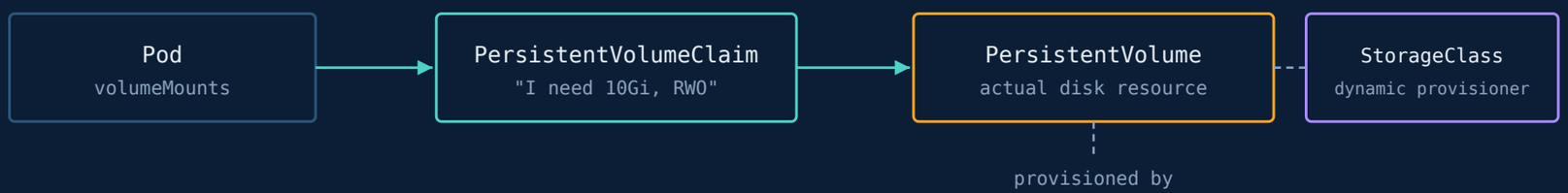
Both decouple configuration from the container image. Same shape, different intent — Secrets are for sensitive values and are base64-encoded (not encrypted by default; enable encryption-at-rest or use a vault/external-secrets integration).

CONFIGMAP.YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  FEATURE_FLAG: "true"
---
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
stringData:              # plain text in, base64 stored
  DB_PASSWORD: "changeme"
```

Both can be injected as environment variables or mounted as files (a volume of one file per key) — mounted-file is preferable for values that change, since Secrets/ConfigMaps mounted as volumes update in place without a pod restart (env vars do not).

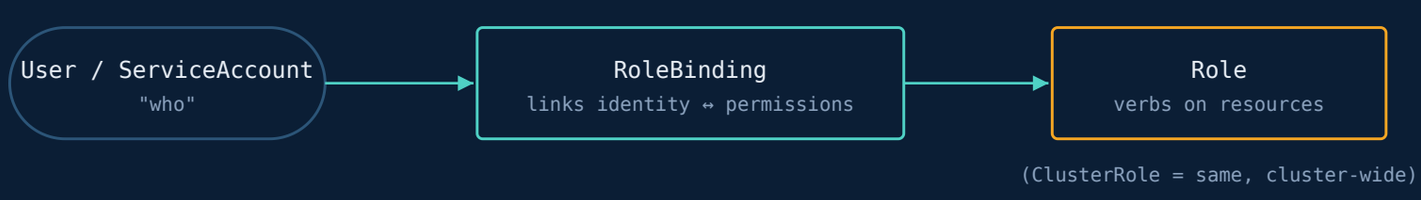
K8S–08 Storage: Volumes, PV, PVC



- **Volume** — storage attached to a pod's lifecycle; simplest form (emptyDir) dies with the pod.
- **PersistentVolume (PV)** — a piece of storage provisioned in the cluster, independent of any pod.
- **PersistentVolumeClaim (PVC)** — a pod's request for storage ("I need 10Gi, ReadWriteOnce"); Kubernetes binds it to a matching PV.
- **StorageClass** — lets PVCs be fulfilled *dynamically* — no PV needs to pre-exist; the class's provisioner creates one on demand.
- Access modes: `ReadWriteOnce` (one node), `ReadOnlyMany`, `ReadWriteMany` (needs a networked filesystem like NFS/EFS).

K8S–09 Namespaces & RBAC

Namespaces partition one cluster into virtual clusters — for teams, environments, or tenants. **RBAC** decides who can do what, where.



KIND	SCOPE
`Role + RoleBinding`	Namespace-scoped permissions
`ClusterRole + ClusterRoleBinding`	Cluster-wide permissions (or reusable across namespaces)

PRINCIPLE

Grant the narrowest Role that does the job, bound in the narrowest namespace. Avoid ClusterRoleBinding to `cluster-admin` for anything but real cluster operators.

K8S–10 Scheduling Controls

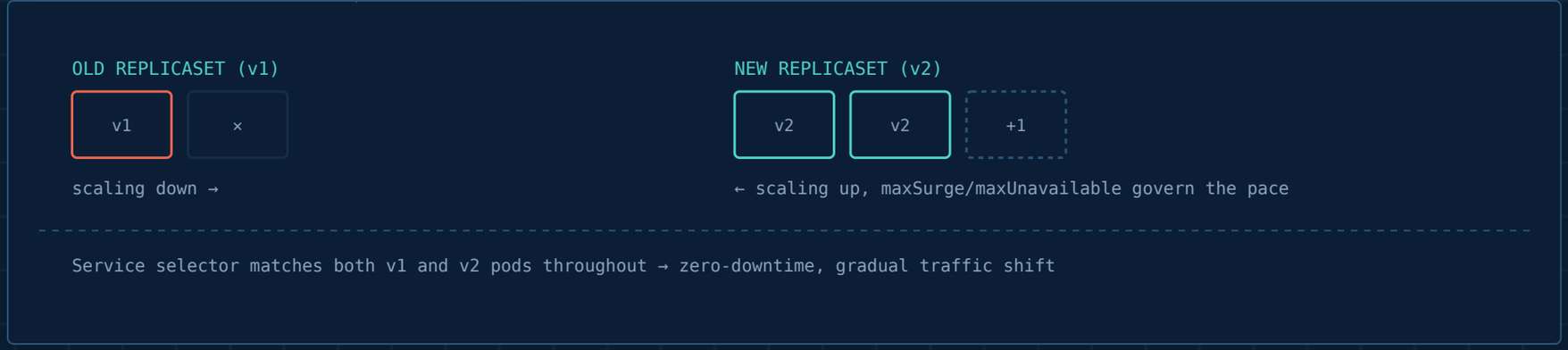
The scheduler's default placement can be steered — pull pods toward nodes, or push them away.

MECHANISM	DIRECTION	WHAT IT DOES
`nodeSelector`	Pod → Node	Simplest match: pod requires a node with an exact label.
`nodeAffinity`	Pod → Node	Richer rules (required/preferred, in/notin) for node labels.
`podAffinity / podAntiAffinity`	Pod → Pod	Co-locate pods together, or keep them apart (e.g. spread replicas across zones).
`Taints (on node)`	Node repels Pods	Node refuses pods unless they carry a matching Toleration.
`Tolerations (on pod)`	Pod tolerates Taint	Lets a pod land on an otherwise-repelling node (e.g. GPU or spot nodes).
`resources.requests / limits`	Capacity fit	Scheduler places pods only on nodes with enough free requested capacity.

RULE OF THUMB

Taints keep pods *out* by default; affinity pulls pods *in* by preference. Use taints for "this node is special" (GPUs, dedicated hardware) and affinity for "these pods like/dislike each other."

K8S–11 Rollout Strategies



STRATEGY	HOW	TRADE-OFF
RollingUpdate (default)	Gradually swap old pods for new, controlled by maxSurge/maxUnavailable	No downtime, but old & new versions briefly coexist
Recreate	Kill all old pods, then start all new ones	Simple, but causes a downtime window — fine for non-serving batch-style apps
Blue-Green	Deploy v2 fully alongside v1, flip Service selector at once	Instant rollback, but 2× resources during the switch
Canary	Route a small % of traffic to v2 (via Ingress/mesh), expand gradually	Safest for risk, but needs extra tooling to split traffic precisely

K8S–12

Autoscaling

AUTOSCALER	SCALES	TRIGGER
HPA (Horizontal Pod Autoscaler)	Number of pod replicas	CPU/memory or custom metrics vs target
VPA (Vertical Pod Autoscaler)	A pod's requests/limits	Historical usage; usually not combined live with HPA on the same metric
Cluster Autoscaler	Number of nodes	Unschedulable pods (scale up) / underused nodes (scale down)

HPA.YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
```

```
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
```

K8S–13

kubectl Cheat Sheet

INSPECT

```
# cluster & context
kubectl cluster-info
kubectl config get-contexts
kubectl config use-context <name>

# get / describe
kubectl get pods -n <ns> -o wide
kubectl get all -n <ns>
kubectl describe pod <name>
kubectl get events --sort-by=.lastTimestamp

# logs & shell
kubectl logs -f <pod> -c <container>
kubectl exec -it <pod> -- sh
```

MUTATE

```
# apply / manage
kubectl apply -f manifest.yaml
kubectl delete -f manifest.yaml
kubectl rollout status deploy/<name>
kubectl rollout undo deploy/<name>
kubectl scale deploy/<name> --replicas=5

# debug
kubectl top pod
kubectl port-forward svc/<name> 8080:80
kubectl get pod <name> -o yaml
kubectl explain deployment.spec
```

alias k=kubectl · -o yaml / -o json · --dry-run=client -o yaml · --watch · --all-namespaces / -A